

1. Pandas DataFrame-Based Assessment Approach

These questions usually check whether trainees can work with tabular data using pandas.

They commonly test:

- creating a DataFrame from raw list data
- adding new columns
- filtering rows based on conditions
- grouping records
- counting records
- merging grouped results
- calculating percentages
- creating summary tables using `crosstab` or `pivot_table`

The most important thing is to understand that **each function has one clear responsibility**. Trainees should not try to write all logic inside one function.

Function 1: Create DataFrame

How to Approach

This is usually the simplest function.

The input will be a list of records. The task is to convert it into a pandas DataFrame using the exact column names given in the question.

General approach:

```
df = pd.DataFrame(data, columns=[...])  
return df
```

Where Trainees Need to Focus

Focus mainly on:

- using the correct number of columns
- using the exact column names given in the question
- keeping date-like values as strings if the instruction says so
- returning the DataFrame

Common Mistakes

Trainees often make these mistakes:

- spelling column names incorrectly
- changing column names to their own preferred names
- creating the DataFrame without column names
- converting date strings into datetime even when not asked
- forgetting to return the DataFrame

Wrong approach:

```
df = pd.DataFrame(data)
```

This creates default column names like 0, 1, 2, 3, which usually breaks the next functions.

Function 2: Monthly Rate or Percentage Calculation

This is often the most important function in pandas assessments.

How to Approach

Break the function into small steps:

1. Make a copy of the DataFrame.
2. Extract the month from the date string.
3. Count total records per entity and month.
4. Filter only the required successful/status records.
5. Count filtered records per entity and month.
6. Merge total count and filtered count.
7. Replace missing values with 0.
8. Calculate the percentage.
9. Return only the expected columns.

General structure:

```
df = df.copy()
```

```
df["Month"] = df["date_column"].str[:7]
```

```
total_df = df.groupby(["entity_column", "Month"]).size().reset_index(name="Total")
```

```
filtered_df = df[df["status_column"] == "required_status"]
```

```
## Alternate method
```

```

filtered_df=df[df['ReturnStatus']=='Returned']

monthly_return_df=filtered_df.groupby(['MemberID','Month']).size().reset_index(name='total')

filtered_count_df = filtered_df.groupby(["entity_column",
"Month"]).size().reset_index(name="Filtered Count")

result = pd.merge(total_df, filtered_count_df, on=["entity_column", "Month"], how="left")

result["Filtered Count"] = result["Filtered Count"].fillna(0)

result["Rate"] = (result["Filtered Count"] / result["Total"]) * 100

return result[["entity_column", "Month", "Rate"]]

```

Where Trainees Need to Focus

The key idea is to understand the difference between:

Total records

and

Filtered records

For example:

- total records means all rows for that entity and month
- filtered records means only rows matching the required condition

Both are needed to calculate a correct percentage.

Common Mistakes

Trainees commonly make these mistakes:

- counting only filtered records and forgetting total records
- using the wrong condition value
- using lowercase when the data contains uppercase, or vice versa
- forgetting `fillna(0)` after merge
- using an inner merge instead of a left merge
- forgetting `reset_index()`
- returning extra helper columns that are not expected
- using `.dt.month` when the instruction says the date should remain a string

Important point:

```
df["date_column"].str[:7]
```

is commonly used when the date is stored as a string and the required output is year-month format.

Function 3: Add Flag Column

This type of function adds a new column with **1** or **0** depending on a condition.

How to Approach

Use a boolean comparison and convert it to integer.

General structure:

```
df = df.copy()
```

```
df["flag_column"] = (df["status_column"] == "required_status").astype(int)
```

```
return df
```

Where Trainees Need to Focus

Focus on:

- checking the correct status/category value
- creating the exact new column requested
- converting boolean values to integers
- returning the updated DataFrame

Common Mistakes

Trainees often:

- return **True** and **False** instead of **1** and **0**
- forget **.astype(int)**
- compare against the wrong text
- create the wrong output column name
- modify the wrong existing column

Wrong approach:

```
df["flag_column"] = df["status_column"] == "required_status"
```

This returns boolean values. Many assessments expect integer flags.

Correct approach:

```
df["flag_column"] = (df["status_column"] == "required_status").astype(int)
```

Function 4: Filter High Count Records

This function usually finds entities that cross a given count threshold.

How to Approach

Steps:

1. Filter rows based on the required condition.
2. Group by the entity column.
3. Count records.
4. Rename the count column.
5. Keep only records that satisfy the threshold condition.
6. Reset index and return.

General structure:

```
filtered_df = df[df["status_column"] == "required_status"]
```

```
result = filtered_df.groupby("entity_column").size().reset_index(name="Count Column")
```

```
result = result[result["Count Column"] > threshold]
```

```
return result.reset_index(drop=True)
```

Where Trainees Need to Focus

Read the wording carefully.

If the question says:

more than n

use:

> threshold

If the question says:

at least n

use:

`>= threshold`

This small difference is one of the most common reasons test cases fail.

Common Mistakes

Trainees commonly:

- count all rows instead of filtering first
- group by the wrong column
- return a Series instead of a DataFrame
- forget to rename the count column
- use `>=` when the question says more than
- use `>` when the question says at least
- forget `reset_index()`

Wrong approach:

```
result = df.groupby("entity_column").size()
```

This counts all records and returns a Series.

Better approach:

```
filtered_df = df[df["status_column"] == "required_status"]
result = filtered_df.groupby("entity_column").size().reset_index(name="Count Column")
```

Function 5: Category-Wise Summary Table

This function usually creates a summary table showing how many times each status/category appears under each group.

How to Approach

Use `pd.crosstab()` or `pivot_table()`.

General structure using `crosstab`:

```
result = pd.crosstab(df["group_column"], df["status_column"])
```

```
result.columns.name = None
```

```
result = result.reindex(columns=["status_1", "status_2", "status_3"], fill_value=0)
```

```
return result.reset_index()
```

Where Trainees Need to Focus

The output should contain all expected status columns, even if some status does not appear in the input data.

That is why `reindex()` is important.

Common Mistakes

Trainees often:

- forget to use `reindex()`
- miss columns when a category is absent in the input
- forget `fill_value=0`
- forget `reset_index()`
- leave unwanted column index names
- use the wrong row or column field in `crosstab`

A simple `crosstab` may pass one sample test but fail hidden tests if one expected category is missing.

2. Dictionary and String-Processing Assessment Approach

These questions usually check whether trainees can process text/list input and use dictionaries correctly.

They commonly test:

- cleaning input
- extracting useful values
- converting text to lowercase
- removing unwanted characters
- looping through lists
- counting frequencies using dictionaries
- finding the most frequent item
- filtering dictionary values by threshold

Again, each function has a separate responsibility.

Function 1: Clean, Extract, or Tokenize Input

This function prepares raw input for counting.

How to Approach

The function may ask trainees to:

- convert text to lowercase
- remove punctuation or unwanted characters
- split text into smaller parts
- extract only valid items
- ignore invalid values

General text-cleaning structure:

```
text = text.lower()
```

```
text = re.sub(r'^\w\s]', "", text)
```

```
items = text.split()
```

```
return items
```

General list-cleaning structure:

```
cleaned_items = []
```

```
for item in input_list:
```

```
    if valid_condition:
```

```
        cleaned_items.append(cleaned_value)
```

```
return cleaned_items
```

Where Trainees Need to Focus

Focus on the order of operations.

Usually the correct order is:

1. normalize input
2. remove unwanted characters
3. split or extract values
4. return a list

Common Mistakes

Trainees often:

- split before cleaning punctuation
- forget lowercase conversion
- forget to import `re` when regex is required
- use the wrong regex pattern

- return a string instead of a list
- include invalid values that should be ignored
- remove too much data while cleaning

If punctuation is not removed properly, values that should be same may be treated differently.

For example:

item
item!

These may become two different values if cleaning is not done.

Function 2: Compute Frequency

This function counts how many times each item appears.

How to Approach

Use an empty dictionary and update counts inside a loop.

General structure:

```
freq_dict = {}
```

```
for item in items:
```

```
    freq_dict[item] = freq_dict.get(item, 0) + 1
```

```
return freq_dict
```

Where Trainees Need to Focus

Understand this part:

```
freq_dict.get(item, 0)
```

It means:

- if the item already exists, use its current count
- if it does not exist, start from 0

Then add 1.

Common Mistakes

Trainees often:

- reset the dictionary inside the loop
- use a list instead of a dictionary
- forget to increment the count
- use direct increment before creating the key
- forget to return the dictionary

Wrong approach:

```
freq_dict = {}
```

```
for item in items:
```

```
    freq_dict = {}
```

```
    freq_dict[item] = 1
```

This resets the dictionary on every loop.

Correct approach:

```
freq_dict = {}
```

```
for item in items:
```

```
    freq_dict[item] = freq_dict.get(item, 0) + 1
```

Function 3: Get Most Frequent Item

This function returns the item with the highest frequency and its count.

How to Approach

First handle empty dictionary.

```
if not freq_dict:
```

```
    return None
```

Then use `max()` with dictionary items.

```
return max(freq_dict.items(), key=lambda x: x[1])
```

Where Trainees Need to Focus

`freq_dict.items()` gives pairs like:

```
(item, count)
```

`lambda x: x[1]` tells Python to compare using the count.

Common Mistakes

Trainees often:

- forget to handle empty dictionary
- return only the item
- return only the count
- use `max(freq_dict)`, which compares keys instead of values
- use the wrong lambda index

Wrong approach:

```
return max(freq_dict)
```

This gives the maximum key, not the most frequent item.

Correct approach:

```
return max(freq_dict.items(), key=lambda x: x[1])
```

Function 4: Filter by Minimum Frequency

This function returns only items whose count satisfies a minimum frequency condition.

How to Approach

Create a new dictionary and add matching items.

General structure:

```
result = {}
```

```
for item, count in freq_dict.items():
```

```
    if count >= n:
```

```
        result[item] = count
```

```
return result
```

Where Trainees Need to Focus

Read the condition carefully.

If the question says:

at least n times

use:

```
>= n
```

If the question says:

more than n times

use:

`> n`

Common Mistakes

Trainees often:

- use `>` instead of `>=`
- use `>=` instead of `>`
- compare the key instead of the count
- return a list instead of a dictionary
- return `None` when no item matches
- forget to use `.items()`

Wrong approach:

```
for item in freq_dict:
    if item >= n:
        result[item] = freq_dict[item]
```

Here `item` is the key, not the count.

Correct approach:

```
for item, count in freq_dict.items():
    if count >= n:
        result[item] = count
```

Final Guidance for Trainees

For pandas questions, focus on:

- exact column names from the question
- filtering before grouping
- using `groupby().size().reset_index(name=...)`
- using `merge()` correctly
- using `fillna(0)` after merging counts
- using `reindex()` in summaries
- returning only expected columns

For dictionary/string questions, focus on:

- cleaning input before counting
- using lowercase where required
- using dictionary `.get()`
- returning the correct data type
- handling empty input
- understanding `>` vs `>=`

Most failures happen because of small mistakes: wrong return type, wrong comparison operator, missing `reset_index()`, missing `fillna(0)`, incorrect spelling, or returning extra/unexpected columns.